

sheet04-programming

November 13, 2025

1 Weighted K-Means Clustering

In this exercise we will simulate finding good locations for production plants of a company in order to minimize its logistical costs. In particular, we would like to place production plants near customers so as to reduce shipping costs and delivery time.

We assume that the probability of someone being a customer is independent of its geographical location and that the overall cost of delivering products to customers is proportional to the squared Euclidean distance to the closest production plant. Under these assumptions, the K-Means algorithm is an appropriate method to find a good set of locations. Indeed, K-Means finds a spatial clustering of potential customers and the centroid of each cluster can be chosen to be the location of the plant.

Because there are potentially millions of customers, and that it is not scalable to model each customer as a data point in the K-Means procedure, we consider instead as many points as there are geographical locations, and assign to each geographical location a weight w_i corresponding to the number of inhabitants at that location. The resulting problem becomes a weighted version of K-Means where we seek to minimize the objective:

$$J(c_1, \dots, c_K) = \frac{\sum_i w_i \min_k \|x_i - c_k\|^2}{\sum_i w_i},$$

where c_k is the k th centroid, and w_i is the weight of each geographical coordinate x_i . In order to minimize this cost function, we iteratively perform the following EM computations:

- **Expectation step:** Compute the set of points associated to each centroid:

$$\forall 1 \leq k \leq K : \quad \mathcal{C}(k) \leftarrow \left\{ i : k = \arg \min_k \|x_i - c_k\|^2 \right\}$$

- **Minimization step:** Recompute the centroid as a the (weighted) mean of the associated data points:

$$\forall 1 \leq k \leq K : \quad c_k \leftarrow \frac{\sum_{i \in \mathcal{C}(k)} w_i \cdot x_i}{\sum_{i \in \mathcal{C}(k)} w_i}$$

until the objective $J(c_1, \dots, c_K)$ has converged.

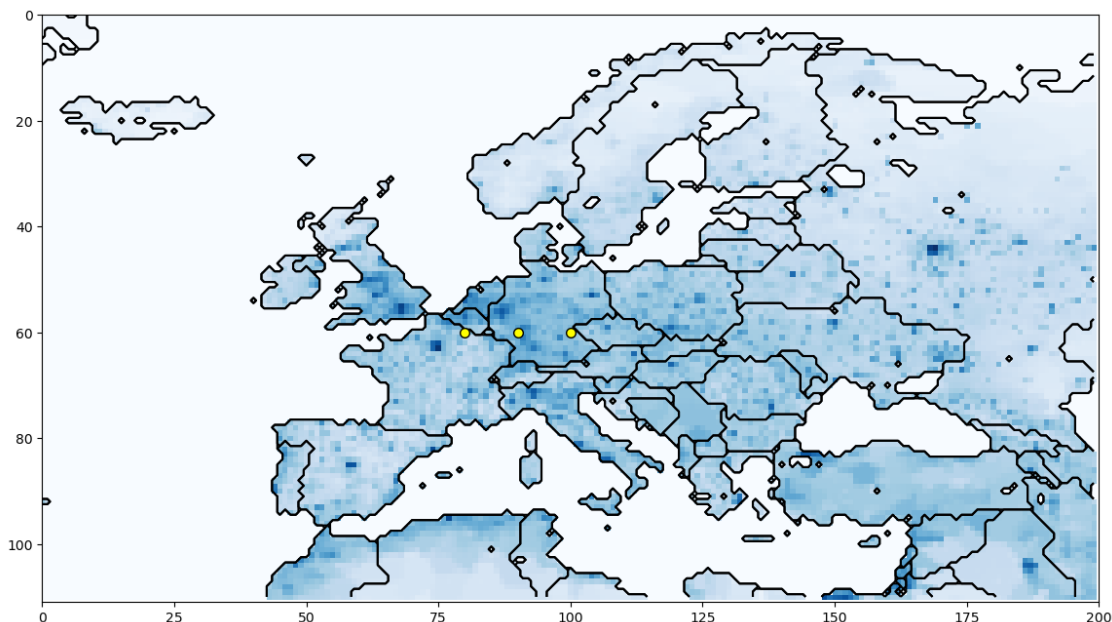
1.1 Getting started

In this exercise we will use data from <http://sedac.ciesin.columbia.edu/>, that we store in the files `data.mat` as part of the zip archive. The data contains for each geographical coordinates (latitude and longitude), the number of inhabitants and the corresponding country. Several variables and methods are provided in the file `utils.py`:

- `utils.population` A 2D array with the number of inhabitants at each latitude/longitude.
- `utils.plot(latitudes,longitudes)` Plot a list of centroids given as geographical coordinates in overlay to the population density map.

The code below plots three factories (white squares) with geographical coordinates (60,80), (60,90),(60,100) given as input.

```
[18]: import utils, numpy
import numpy as np
%matplotlib inline
utils.plot([60,60,60],[80,90,100])
```



Also, to get a dataset of geographical coordinates associated to the image given as an array, we can use:

```
[19]: x,y = numpy.indices(utils.population.shape)
locations = numpy.array([x.flatten(),y.flatten()]).T
```

1.2 Initializing Weighted K-Means (25 P)

Because K-means has a non-convex objective, choosing a good initial set of centroids is important. Centroids are drawn from the following discrete probability distribution:

$$P(x, y) = \frac{1}{Z} \cdot \text{population}(x, y)$$

where Z is a normalization constant. Furthermore, to avoid identical centroids, we add a small Gaussian noise to the location of centroids, with standard deviation 0.01.

Task:

- Implement the initialization procedure above.

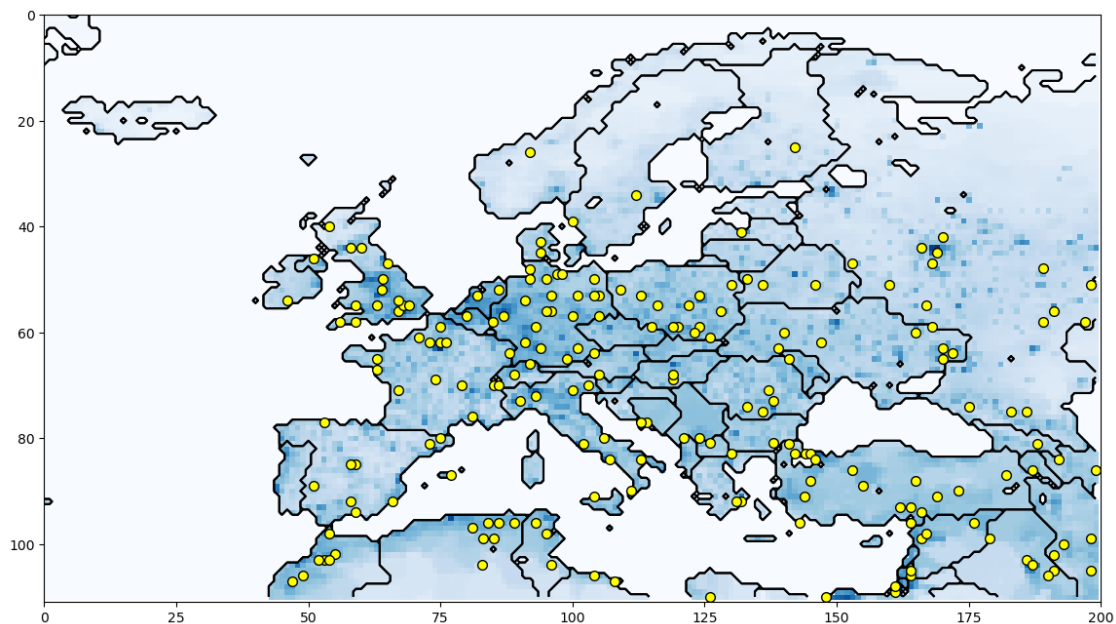
```
[20]: def initialize(K, population):
    # YOUR CODE HERE
    x, y = numpy.indices(population.shape)
    locations = numpy.array([x.flatten(), y.flatten()]).T
    weights = population.flatten().astype(float)
    weights /= weights.sum()

    indices = numpy.random.choice(len(weights), size=K, replace=False,
    ↪p=weights)
    centroids = locations[indices]

    centroids = centroids + numpy.random.normal(0, 0.01, size=centroids.shape)
    return centroids
    # -----
```

The following code runs the initialization procedure for $K=200$ clusters and visualizes the centroids obtained with the initialization procedure using `utils.plot`.

```
[21]: centroids_init = initialize(200, utils.population)
utils.plot(centroids_init[:,0], centroids_init[:,1])
```



1.3 Implementing Weighted K-Means (75 P)

Task:

- Implement the weighted K-Means algorithm. Your algorithm should run for `nbit` iterations and print the value of the objective after training. If verbose, it should also print the value of the objective at each iteration.

```
[22]: def wkmeans(centroids, points, weights, verbose, nbit):
    # YOUR CODE HERE
    centroids = centroids.astype(float).copy()
    points = points.astype(float)
    weights = weights.astype(float)

    N, d = points.shape
    K = centroids.shape[0]
    total_w = weights.sum()

    for it in range(1, nbit+1):

        #Expectation step
        diff = points[:, None, :] - centroids[None, :, :]
        d2 = (diff**2).sum(axis=2)

        labels = np.argmin(d2, axis=1)

        #Objective J
        idx = np.arange(N)
        dist_to_own = d2[idx, labels]
        J_num = (weights * dist_to_own).sum()
        J = J_num / total_w

        if verbose:
            print(f"it = {it:2d}: J = {J:7.2f}")
        elif it == nbit:
            #if verbose=false, print last iteration
            print(f"it = {it:2d}: J = {J:7.2f}")

        #Minimization step
        for k in range(K):
            mask = (labels == k)
            if not np.any(mask):
                #empty Cluster
                new_idx = np.random.choice(N, p=weights/total_w)
                centroids[k] = points[new_idx]
```

```

        else:
            wk = weights[mask]
            pk = points[mask]
            centroids[k] = (wk[:, None] * pk).sum(axis=0) / wk.sum()

    return centroids
# -----

```

The following code runs the weighted k-means on this data, and displays the final centroids.

```

[23]: weights = utils.population.flatten()*1.0

centroids = wkmeans(centroids_init, locations, weights, True, 50)

utils.plot(centroids[:,0], centroids[:,1])

```

```

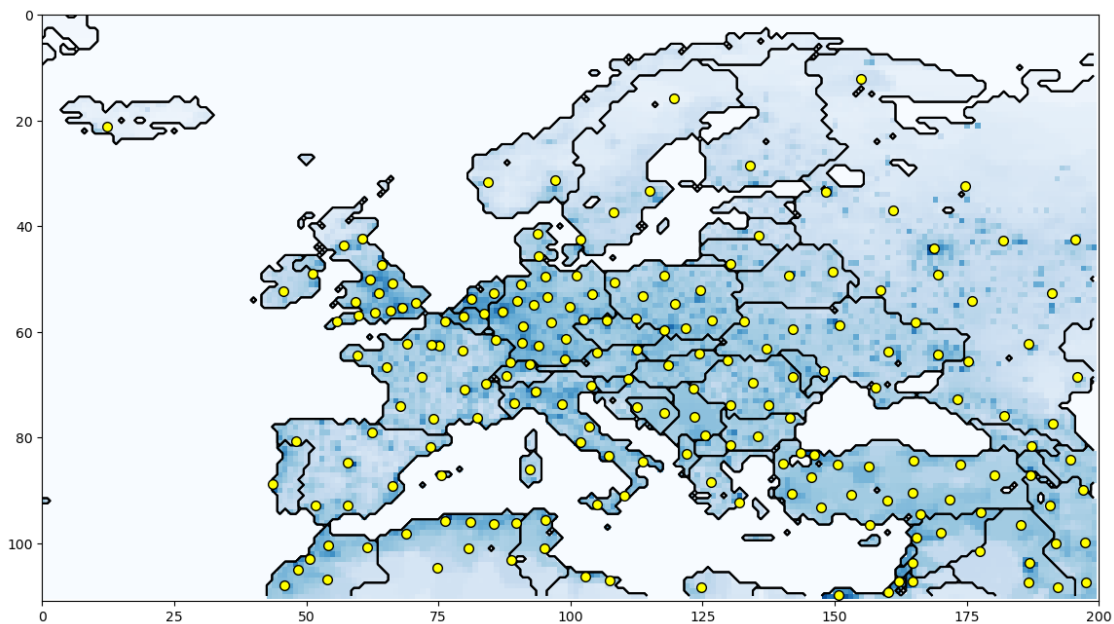
it = 1: J = 16.40
it = 2: J = 10.77
it = 3: J = 9.07
it = 4: J = 8.39
it = 5: J = 7.91
it = 6: J = 7.63
it = 7: J = 7.49
it = 8: J = 7.40
it = 9: J = 7.33
it = 10: J = 7.27
it = 11: J = 7.23
it = 12: J = 7.21
it = 13: J = 7.17
it = 14: J = 7.13
it = 15: J = 7.01
it = 16: J = 6.80
it = 17: J = 6.75
it = 18: J = 6.73
it = 19: J = 6.73
it = 20: J = 6.73
it = 21: J = 6.73
it = 22: J = 6.73
it = 23: J = 6.73
it = 24: J = 6.73
it = 25: J = 6.73
it = 26: J = 6.73
it = 27: J = 6.73
it = 28: J = 6.73
it = 29: J = 6.73
it = 30: J = 6.73
it = 31: J = 6.73
it = 32: J = 6.73

```

```

it = 33: J = 6.73
it = 34: J = 6.73
it = 35: J = 6.73
it = 36: J = 6.73
it = 37: J = 6.73
it = 38: J = 6.73
it = 39: J = 6.73
it = 40: J = 6.73
it = 41: J = 6.73
it = 42: J = 6.73
it = 43: J = 6.73
it = 44: J = 6.73
it = 45: J = 6.73
it = 46: J = 6.73
it = 47: J = 6.73
it = 48: J = 6.73
it = 49: J = 6.73
it = 50: J = 6.73

```



Observe that the k-means algorithm is non-convex, and arrives in local optima of different quality depending on the initialization:

```

[25]: for i in range(5):
        wkmeans(initialize(200, utils.population), locations, weights, False, 50)

```

```

it = 50: J = 6.73
it = 50: J = 7.44
it = 50: J = 7.52

```

```
it = 50: J = 7.27  
it = 50: J = 8.43
```

```
[ ]:
```